Explainable AI-Based Fake News Detection and Claim Verification System

Comprehensive Methodological & Architectural Technical Report

Project Batch: R7A	**Subject:** NLP Academic Project	**Date:** October 2026
Student Authors:	Amal Dev M L (Roll No: 215) A S Adityanarayanan (Roll No: 220) Rohit Ajoy (Roll No: 255)	**Core Stack:** LangChain · Sentence-Transformers DeBERTa-v3 NLI · Vector RAG

1. Executive Summary & Abstract Alignment

The exponential surge of digital news and unverified social media narratives has intensified the proliferation of misleading claims. Traditional fact-checking mechanisms are manual, unscalable, and susceptible to latency, while purely generative black-box Large Language Models (LLMs) frequently hallucinate falsehoods without empirical grounding. This project implements an **Explainable AI-Based Fake News Detection and Claim Verification System** that synergistically integrates Natural Language Processing (NLP), semantic embeddings, dense vector search, Retrieval-Augmented Generation (RAG), and a pretrained Natural Language Inference (NLI) model within a unified **LangChain** architecture. The system digests arbitrary articles or claims, extracts the salient factual propositions, queries a curated knowledge base via dense cosine vector search (with dynamic web-scale Wikipedia RAG fallback), and classifies the stance into one of three verdicts: **Supported**, **Refuted**, or **Insufficient Evidence** alongside rigorous confidence scores and human-interpretable Explainable AI (XAI) feature attributions.

2. End-to-End System Architecture

The system is architected as a modular, high-throughput pipeline where each stage is decoupled, reproducible, and verifiable. The LangChain framework acts as the connective backbone orchestrating state propagation between components.

Phase	Component	Methodology / Model	Output Artifact
Phase 1	Claim Extraction	Sentence boundary detection, capitalized phrase clustering, stopword pruning	Canonical claim & search query
Phase 2	Vector RAG Retrieval	Dense embeddings via SentenceTransformers (all-MiniLM-L6-v2) + InMemoryVectorStore	Top-k grounded factual evidence (Curated KB)
Phase 3	Dynamic Web Fallback	Wikipedia REST API multi-query synthesis with lexical relevance filtering	Dynamic external evidence & runtime vector indexation
Phase 4	NLI Stance Verification	DeBERTa-v3-base-mnli-fever-anli (Disentangled Attention Transformer)	Verdict (Supported / Refuted / Insufficient) & Probabilities
Phase 5	Explainable AI (XAI)	Salient token alignment attribution & human-readable rationale synthesis	XAI word weights + contextual summary
Phase 6	Presentation Layer	Flask REST API, asynchronous warm-up daemon, and interactive reactive UI	Rendered verification dashboard

3. Methodology 1: NLP Claim Extraction & Linguistic Preprocessing

Articles submitted by users often contain extraneous stylistic preamble, journalistic attribution, or non-factual rhetorical framing. The system applies a deterministic NLP pipeline implemented in detector.extract_claim_and_keywords:

A. Sentence Boundary Detection & Proposition Isolation: The input text is cleaned and segmented using lookahead regular expressions r'(?<=[.!?])\s+'. The first 1–2 sentences are captured as the core hypothesis claim, preserving syntactic completeness.

B. Capitalized Entity Harvesting & Stopword Pruning: Named entities and multi-word proper nouns are extracted using r'\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)*\b'. Tokens are evaluated against an extensive stopword lexicon (64+ grammatical particles) to filter noise while preserving crucial substantive nouns, verbs, and numerical values.

C. Multi-Clause Decomposition & Synonym Augmentation: Complex compound claims with coordinating conjunctions (*and, but, or, whereas*) are split into distinct sub-clauses. A domain-specific synonym index (e.g., *round* ↔ *spherical, revolves* ↔ *orbits, cure* ↔ *treatment*) expands lexical coverage, enabling robust recall across variant linguistic formulations.

4. Methodology 2: Multi-Source Curated Knowledge Base & Vector Search (RAG)

Relying exclusively on general-purpose encyclopedias like Wikipedia presents severe limitations: Wikipedia lacks dedicated articles for viral social media hoaxes, specific political speeches, and emergent misinformation. To address this, our system introduces a **Multi-Source Curated Knowledge Base** combining verified records from the **LIAR Dataset (PolitiFact)**, **Snopes Fact-Check Repository**, and authoritative scientific encyclopedias. Dense vector retrieval in rag_pipeline.py indexes these multi-source documents to enable semantic evidence retrieval.

A. Integration of LIAR & PolitiFact Fact-Checks: The LIAR benchmark (Wang, ACL 2017) provides 12,836 real-world statements evaluated by professional fact-checkers at PolitiFact.com. We ingested key political statements (e.g., presidential rally claims, unemployment statistics, election integrity assertions) along with their authoritative justifications into data/knowledge_base.json. This enables verifiable fact-checking on political and policy discourse.

B. Integration of Snopes & Viral Rumor Debunkings: To counteract widespread online conspiracy theories (e.g., 5G coronavirus transmission, vaccine microchip myths, hazardous chemical cures), verified debunkings from Snopes and public health bodies are indexed with detailed scientific refutations.

C. Dense Semantic Embedding Architecture: We utilize the **Sentence-Transformers all-MiniLM-L6-v2** model, fine-tuned on over 1 billion sentence pairs for semantic similarity. It maps textual statements into a continuous 384-dimensional dense vector space $\mathbf{z} \in \mathbb{R}^{384}$. Precomputed embeddings are persisted in data/vector_cache.pkl, reducing vector store initialization to under **0.01 seconds**.

B. Mathematical Similarity Metric: Given a query vector $\mathbf{u}$ and an indexed knowledge document vector $\mathbf{v}$, their semantic proximity is calculated using **Cosine Similarity**:

```
Cosine Similarity(u, v) = (u · v) / (||u||■ · ||v||■) = ∑(u■ · v■) / ( √(∑ u■²) · √(∑ v■²) ) Score Range: [-1.0, 1.0], where 1.0 indicates identical directional semantic alignment.
```

C. Curated Knowledge Base Indexing: We engineered a structured reference database in data/knowledge_base.json containing ground-truth encyclopedic entries across Astronomy, Geography, Medical Sciences, History, and Debunked Pseudoscience. At application startup, each record is ingested as a LangChain Document object with rich metadata (ID, title, category, verified source URL) and indexed into LangChain's InMemoryVectorStore. Vector search operates in sub-millisecond latency on CPU.

5. Methodology 3: Multi-Tier Hybrid Retrieval & Dynamic Web Fallback

Real-world fact checking cannot rely exclusively on a static local corpus, nor can it trust unvetted web scrapers. We implement a three-tier hierarchical retrieval strategy in detector._chain_vector_retrieve:

Tier 1: Curated Vector Store Query: The extracted claim is embedded and matched against the local knowledge base. If the top result yields $\text{Similarity} \ge 0.45$, the local evidence is selected with guaranteed empirical accuracy.

Tier 2: Dynamic Web-Scale Wikipedia RAG: If the local vector score falls below 0.45, the pipeline initiates a dynamic fallback to the official Wikipedia REST API. It executes multi-clause queries, enforces antonym penalties (e.g., discarding 'Flat Earth' when the claim does not advocate it), discards disambiguation hubs, and scores candidate snippets based on keyword density. The selected article is then dynamically converted into a LangChain Document and indexed into the vector store for real-time semantic caching.

Tier 3: Model-Only Zero-Shot Fallback: In edge cases where neither local knowledge nor Wikipedia contains sufficient data, the system encapsulates the claim within a neutral fact-checking premise, allowing the pretrained NLI model to render an assessment grounded in its internal representation while explicitly notifying the user via UI warning badges.

6. Methodology 4: Natural Language Inference (NLI) & Stance Classification

Fact verification is mathematically framed as a **Natural Language Inference (NLI)** problem between two textual sequences:
• **Premise (P):** The retrieved factual evidence document.
• **Hypothesis (H):** The claim extracted from the input article.

A. Deep Bidirectional Transformer Backbone: We deploy **MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli**. DeBERTa (Decoding-enhanced BERT with Disentangled Attention) introduces two fundamental algorithmic breakthroughs over standard BERT/RoBERTa:
1. *Disentangled Attention:* Each word is represented by two separate vectors encoding its content and its relative position, computing cross-attention as four components (content-to-content, content-to-position, position-to-content, position-to-position).
2. *Enhanced Masked Language Modeling:* Incorporates absolute positions in the decoding layer to predict masked tokens.

B. Multi-Task Supervised Fine-Tuning: The model is pretrained on MNLI (Multi-Genre NLI), ANLI (Adversarial NLI), and critically **FEVER** (Fact Extraction and VERification), a benchmark of 185,000+ Wikipedia-grounded factual claims. This endows the model with an inductive bias tailored for truth-conditional verification.

C. Mathematical Probability Distribution & Verdict Mapping: Given sequence $[CLS] P [SEP] H [SEP]$, the final hidden state of the $[CLS]$ token passes through a linear classification head producing unnormalized logits $\mathbf{z} = [z_0, z_1, z_2]$. Softmax normalization yields the probability distribution:

```
P(y = c | P, H) = exp(z_c) / ∑(exp(z_j)) for c ∈ {0: Entailment, 1: Neutral, 2: Contradiction} • Entailment (y = 0) ■■■ VERDICT: SUPPORTED (High semantic agreement with evidence) • Contradiction (y = 2) ■■■ VERDICT: REFUTED (Empirically negates or conflicts with evidence) • Neutral (y = 1) ■■■ VERDICT: INSUFFICIENT DATA (Evidence lacks decisive grounds to prove/disprove)
```

7. Methodology 5: Explainable AI (XAI) & Interpretability Techniques

A critical flaw in modern machine learning systems is the opacity of 'black-box' decisions. For high-stakes fact checking, users must understand *why* a particular verdict was assigned. We incorporate a multi-faceted XAI architecture:

A. Salient Feature Attribution & Lexical Overlap: Implemented in _fast_keyword_highlights, the system analyzes content token co-occurrences between the hypothesis and the premise. Tokens confirmed in the evidence are weighted positively

(supports_verdict, $+1.0$), while tokens asserted in the claim but absent from the evidence are attributed divergent influence (against_verdict, -0.3). This runs in sub-millisecond execution time, eliminating the high computational penalty of standard CPU perturbation loops.

B. Leave-One-Out (LOO) Sensitivity Ablation: For deep analysis, our implementation provides exact LOO ablation where individual content tokens w_i are masked and the model re-evaluated in a single batched tensor pass. The attribution score is calculated as $\Delta_i = P(\text{pred})_{orig} - P(\text{pred})_{\setminus w_i}$.

C. Contextual Human-Readable Rationale Generation: Rather than displaying raw tensors, the system dynamically compiles a coherent, grammatical explanation synthesizing the retrieval source, similarity percentage, top supporting/conflicting phrases, and final classification confidence.

8. Methodology 6: LangChain Framework Pipeline Orchestration

To satisfy the core requirement of the project abstract, the entire lifecycle is unified under the **LangChain** architecture. In rag_pipeline.build_langchain_verification_chain, we construct a sequential RunnableSequence utilizing LangChain's composable primitives:

```
# LangChain Verification Chain Definition (rag_pipeline.py) verification_chain = (
RunnableLambda(_chain_extract_claim) # Step 1: NLP Proposition Segmentation |
RunnableLambda(_chain_vector_retrieve) # Step 2: Dense Semantic Vector Search (RAG) |
RunnableLambda(_chain_nli_classify) # Step 3: DeBERTa-v3 NLI Stance Evaluation |
RunnableLambda(_chain_explain_and_format) # Step 4: Explainable AI Rationale Synthesis ) # Execution: result
= verification_chain.invoke({'clean_text': text, 'start_time': start})
```

9. Empirical Validation & Case Study Evaluations

The system was evaluated against representative real-world test cases spanning science, geography, political discourse, and public health:

Input Claim	Ground Truth	Retrieved Evidence Source	Vector Sim	Verdict	Confidence
The Earth is round and revolves around the Sun.	True	Curated KB: Shape of the Earth and Solar Orbit	64.3%	**SUPPORTED**	99.9%
The Earth is flat and not a sphere.	False	Curated KB: Flat Earth Claims & Refutation	79.7%	**REFUTED**	99.3%
5G cellular networks transmit the coronavirus virus.	False	Snopes: 5G Mobile Technology & Virus Myth	76.1%	**REFUTED**	99.6%
Widespread software fraud in voting machines altered the 2020 election.	False	PolitiFact (LIAR): Voting Machine Fraud Claim	72.0%	**REFUTED**	98.6%
The Great Wall of China is visible from the Moon with naked eye.	False	Snopes: Great Wall Visibility from the Moon	82.8%	**REFUTED**	98.5%
The Eiffel Tower is located in Paris, France.	True	Curated KB: The Eiffel Tower and Its Location	80.7%	**SUPPORTED**	99.9%

10. Computational Efficiency & Optimization Strategies

Operating deep Transformer models on commodity hardware with 2 CPU cores presents significant latency challenges. Through systematic profiling, we introduced four architectural optimizations that reduced inference latency from ~16 seconds to **3.9–4.7 seconds**:

1. **Premise Salience Windowing:** Rather than encoding unconstrained 500-token documents into quadratic cross-attention matrices, the pipeline isolates the top 2 ground-truth sentences (max_length=192), speeding up DeBERTa inference by over 60%.
2. **Precomputed Vector Store Disk Cache:** Pre-encodes knowledge base embeddings to data/vector_cache.pkl, reducing startup vector indexing to **0.01 seconds**.
3. **Offline Local Execution (local_files_only=True):** Disables redundant remote HTTP API check-ins to HuggingFace Hub on model instantiation, saving 3–5 seconds of startup latency.
4. **Dual-Tier In-Memory Result Caching:** An MD5 query cache delivers instantaneous (<0.01s) idempotent responses for repeated claim verification.

11. Conclusion & Academic References

The developed system demonstrates that reliable, evidence-grounded fact checking is achievable by integrating dense vector semantic search, pretrained Natural Language Inference, and Explainable AI within a unified LangChain pipeline. By replacing ungrounded generative LLM guesses with verifiable evidence retrieval, the pipeline delivers transparent, empirically grounded verdicts essential for mitigating online misinformation.

[1] He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *ICLR 2021*.

[2] Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). FEVER: a large-scale dataset for Fact Extraction and VERification. *NAACL 2018*.

[3] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.

[4] Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

[5] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). 'Why Should I Trust You?': Explaining the Predictions of Any Classifier. *ACM SIGKDD 2016*.

[6] Chase, H. (2022). LangChain: Building applications with LLMs through composability and modular chains.